

PART 2 - Advanced Distributed Systems Expansion

Objective - Assignment (5%)

After completing the basic microservices lab, students must enhance the system using advanced distributed systems concepts and analyze how transparency is achieved in their architecture.

This section transforms the basic microservice application into a more realistic distributed system.

PART 2A - Advanced Feature Expansion

Each group must implement at least **FIVE** advanced distributed systems features from the following list.

Option 1 - Failure Transparency

Objective

Ensure the system continues functioning even if one or more services fail.

Requirements

Students must:

- Stop one running service
- Handle timeout exceptions
- Display fallback response
- Keep aggregator operational

Suggested Enhancements

- Retry mechanism
- Error logging
- Offline service indicator

Expected Outcome

The dashboard should still work even when some services become unavailable.

Option 2 - Replication Transparency

Objective

Run multiple replicas of the same microservice.

Requirements

Students must:

- Create at least 2 replicas of one service
- Use Docker Compose scaling OR Nginx load balancing
- Demonstrate requests being distributed

Suggested Commands

```
docker compose up --scale service1=3
```

Expected Outcome

Users should interact with the service without knowing multiple replicas exist.

Option 3 - Location Transparency

Objective

Deploy services on different machines or cloud platforms.

Requirements

Students must:

- Deploy services in different environments
- Use public URLs
- Keep aggregator functionality unchanged

Example Deployment Locations

- Local machine
- AWS EC2
- GCP Cloud Run
- Another student machine

Expected Outcome

Users should not know where services are physically hosted.

Option 4 - Concurrency Transparency

Objective

Demonstrate simultaneous access by multiple users.

Requirements

Students must:

- Perform concurrent requests
- Test multiple users accessing dashboard
- Observe behavior under load

Suggested Tools

ab
hey
curl

Example

```
ab -n 1000 -c 50 http://localhost:5000/
```

Expected Outcome

System should continue operating for multiple simultaneous users.

Option 5 - Scalability Transparency

Objective

Demonstrate horizontal scalability.

Requirements

Students must:

- Add additional service replicas
- Compare performance before and after scaling
- Document observations

Suggested Enhancements

- Add load balancing
- Measure response time

Expected Outcome

The system should scale without changing client behavior.

Option 6 - Health Monitoring

Objective

Implement service monitoring.

Requirements

Students must:

- Use `/health` endpoints
- Show service status on dashboard
- Detect unavailable services

Suggested Dashboard Indicators

- ONLINE
- OFFLINE
- DEGRADED

Expected Outcome

Aggregator should monitor system health dynamically.

Option 7 - Distributed Logging

Objective

Collect logs from distributed services.

Requirements

Students must:

- Add logging to services
- Log incoming requests
- Log failures and exceptions

Suggested Python Module

logging

Expected Outcome

Students should analyze distributed events through logs.

Option 8 - Artificial Network Delay Simulation

Objective

Understand distributed latency.

Requirements

Students must:

- Add artificial delays
- Observe timeout behavior
- Analyze user experience impact

Example

```
time.sleep(5)
```

Expected Outcome

Students should understand latency and timeout handling.

PART 2B – System Analysis Questions

Transparency Analysis Questions

You must answer the following questions based on their implementation and observations during the lab activities.

If your implementation does not currently include features related to a specific type of transparency, explain:

- why the transparency is not fully achieved in the current system, and
- how the system could be modified or expanded to support it.

1. Access Transparency

Question

How does your system provide access transparency?

Guidelines

Explain:

- How users access all services using the same HTTP protocol
- How APIs remain consistent
- How the aggregator hides service complexity

2. Location Transparency

Question

How does your system provide location transparency?

Guidelines

Explain:

- Whether services run on different machines/cloud platforms
- How users are unaware of physical service location
- How URLs or APIs abstract physical deployment

3. Replication Transparency

Question

How does your system provide replication transparency?

Guidelines

Explain:

- Use of multiple replicas
- Load balancing mechanisms
- How users interact with a single logical service

4. Failure Transparency

Question

How does your system handle service failures?

Guidelines

Explain:

- Timeout handling
- Exception handling
- Fallback responses
- Graceful degradation

5. Concurrency Transparency

Question

How does your system support multiple simultaneous users?

Guidelines

Explain:

- Parallel requests
- Multi-user access
- Concurrent request handling

6. Scalability Transparency

Question

How can your system scale to support more users?

Guidelines

Explain:

- Horizontal scaling
- Adding service replicas
- Docker Compose scaling
- Cloud scalability

7. Performance Transparency

Question

How does your system maintain acceptable performance under load?

Guidelines

Explain:

- Load balancing
- Timeout mechanisms
- Efficient communication
- Response handling

8. Persistence Transparency

Question

How could persistence transparency be added to your system?

Guidelines

Discuss:

- Databases
- Persistent storage
- Saving service state
- Recovering after restart

9. Single Point of Failure (SPOF)

Question

Identify possible Single Points of Failure in your architecture.

Guidelines

Discuss:

- Aggregator failure
- Database failure
- Network dependency

Also explain:

- How SPOF can be reduced

10. Distributed Systems Reflection

Question

Why is this application considered a distributed system?

Guidelines

Explain:

- Independent services
- Network communication
- Partial failures
- Multiple nodes
- Distributed deployment

PART 2C - Bonus Advanced Extensions (Optional)

Groups may implement additional features for bonus marks. (Implement max 1 options)

Bonus Option 1 — Kubernetes Deployment

Deploy services using:

- Kubernetes
- Minikube
- K3s

Bonus Option 2 - API Gateway

Implement:

- Nginx reverse proxy
- Central routing
- Request forwarding

Bonus Option 3 - Monitoring Stack

Use:

- Prometheus
- Grafana

Bonus Option 4 - Message Queue Integration

Use:

- RabbitMQ
- or
- Apache Kafka

Deliverables

Each group must submit:

1. Source Code
2. Dockerfiles
3. Docker Compose File
4. Deployment Screenshots
5. Public URLs
6. Transparency Analysis Report
7. Failure Demonstration Evidence
8. Reflection Answers

End of Part 2

ENHANCED DISTRIBUTED MICROSERVICES COLOR DASHBOARD

After Expansion (Part 2)

